

- **What is Data?**

Data is a collection of raw facts and figures about something.

Raw Data → unprocessed facts

Information → Processed Data (organized and meaningful)

Example: Shop Sales

Raw Data (unprocessed):

A company collects daily sales data (for a week):

[120, 150, 100, 130, 170, 90, 110]

Information (Processed and meaningful):

- Average sales per day: 124 units
- Highest sales = 170 units (on Day 5)
- Lowest sales = 90 units (on Day 6)
- Trend: Sales dropped on weekends

- ***Types of Data:***

1. **Qualitative Data:**

Data that describes qualities, categories or names.

For example:

Colors of cars → Red, Blue, Black

Gender → Male, Female

Fruit Names → Apple, Banana, Mango

2. **Quantitative Data:**

Data that can be measured and written in numbers.

For example:

Age → 12 years, 50 years

Height → 160cm, 190 cm

Marks → 80, 70, 65

- **Working With CSV Files:**

A CSV (Comma Separated View) file is file format for storing tabular data (i.e. rows and columns).

Example csv file content.

```
Name, Age, Marks  
Faheem, 20, 85  
Taha, 22, 86  
Raza, 19, 90
```

Python provides multiple ways for working with csv files.

1. Using built-in *csv module*.
2. Using *pandas library*. (a powerful tool for data analysis)

For using csv module, we need to import the module at the start of program.

import csv

- *Reading from CSV File:*

csv.reader() reads each row of a csv file into a list of strings.

```
import csv

# open and read csv file
with open("students_50_records.csv", mode="r") as file:
    reader = csv.reader(file)
    for row in reader:
        print(row)
```

- **Writing to CSV file:**

- Write multiple rows

```
import csv

data = [
    ["Name", "Age", "Marks"],
    ["Ali", 20, 85],
    ["Sara", 21, 90],
]

with open("sample csv file.csv", mode="w", newline="") as file:
    writer = csv.writer(file)
    writer.writerows(data)
```

- Write multiple rows

```
with open("sample csv file.csv", mode="a", newline="") as file:
    writer = csv.writer(file)
    writer.writerow(["Faheem", 24, 90])
```

- ***Reading CSV as Dictionary:***

Easier to access data by column names.

```
with open("sample csv file.csv", mode="r") as file:
    reader = csv.DictReader(file)
    for row in reader:
        print(row)
```

- ***Write CSV from list of Dictionaries:***

```
data = [
    {"Name": "Ali", "Age": 20, "Marks": 85},
    {"Name": "Sana", "Age": 21, "Marks": 90},
]

with open("students.csv", mode="w", newline="") as file:
    fieldnames = ["Name", "Age", "Marks"]
    writer = csv.DictWriter(file, fieldnames=fieldnames)
    writer.writeheader() # Write column names
    writer.writerows(data)
```

- **Modules:**

Modules are files containing Python code that may include functions, classes or variables. Importing modules allows you to access their functionality in your own scripts.

Modules allow you to divide your program in to individual files that can be reused as needed.

1. Standard Modules: (*import math*)
2. Creating and importing your own modules

- **Packages:**

A package is a folder that contains one or more Python modules. It must also contain a special module called `__init__.py`

The `__init__.py` module does not need to contain any code. It only needs to exist so that Python recognizes the package folder as a Python package.

```
mypackage/  
├── __init__.py  
├── module1.py  
└── module2.py
```

Modules from the package are imported using the syntax:

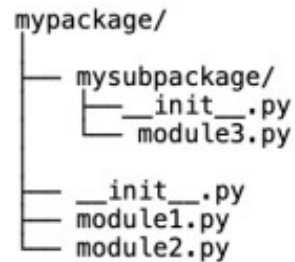
`import <package_name>.<module_name> as <alias>`

Or

`from <package_name> import <module_name> as <alias>`

- **Sub-packages:**

A package nested inside another package is called sub-package.



Modules from sub-packages are imported using syntax.

import <package_name>.<subpackage_name>.<module_name> as <alias>

Or

from <package_name>.<subpackage_name> import <module_name> as <alias>

Note:

Subpackages are great practice for organizing your code inside the very large packages. They help keep the folder structure of a package clean and organized.

However deeply nested **subpackages** introduce long dotted module names. You can imagine how much typing it would take to import the **module** from a **subpackage** of a **subpackage** of a **subpackage** of a **package**.

It is a good practice to try and keep your **subpackages** at most one or two levels deep.

- ***Module and Packages (Task):***

1. Create a project folder called *package_exercise/*, now create a package called *helper/* with three modules: *__init__.py*, *string.py* and *maths.py*.
 - In the *string.py* module, add a function called *shout()* that takes a single string parameter and returns a new string with all of the letters in uppercase.
 - In the *maths.py* module, add a function called *area()* that take two parameters called *length* and *width* and return their product *length x width*.
2. In the root project folder, create a module called *main.py* that import *shout()* and *area()* functions. Use the *shout()* and *area()* functions to print the following output:

THE AREA OF A 5-BY-8 RECTANGLE IS 40